

Ampalis Team

Gruppo 2 · Ingegneria del Software · a.a. 2026/2027

Way of Working

Regole di lavoro adottate da Ampalis Team: strumenti, convenzioni e procedure condivise.

Versione	1.0.0
Data	2026-10-29
Stato	Redatto
Uso	Interno
Destinatari	Ampalis Team, Prof. Tullio Vardanega, Prof. Riccardo Cardin
Redazione	Pietro Vigolo
Verifica	Matteo Cetraro
Approvazione	Ivan Valente

ampalisteam@gmail.com

Registro delle modifiche

Versione	Data	Autore	Verificatore	Descrizione
1.0.0	2026-10-29	Pietro Vigolo	Matteo Cetraro	Rilascio per la candidatura
0.1.0	2026-10-28	Pietro Vigolo	Matteo Cetraro	Prima stesura del documento

Indice

1 Introduzione **2**

1.1 Scopo del documento . . . 2

1.2 Glossario . . . 2

1.3 Riferimenti . . . 2

2 Processi primari **2**

2.1 Fornitura . . . 2

2.2 Sviluppo . . . 3

3 Processi di supporto **3**

3.1 Documentazione . . . 3

3.2 Gestione della configurazione . . . 5

3.3 Verifica . . . 7

3.4 Gestione della qualità e validazione . . . 7

4 Processi organizzativi **7**

4.1 Gestione organizzativa . . . 7

4.2 Formazione . . . 8

1 Introduzione

1.1 Scopo del documento

Il documento raccoglie le regole di lavoro (*way of working*) che Ampalis Team adotta dalla propria costituzione: strumenti, convenzioni e procedure che tutti i componenti applicano allo stesso modo. Ogni regola nasce da una decisione presa in riunione e registrata in un verbale.

Il documento è la base delle *Norme di Progetto*, che lo sostituiranno dopo l'aggiudicazione dell'appalto. Per questo segue già l'organizzazione per processi delle Norme: processi primari, di supporto e organizzativi. Le parti non ancora necessarie sono indicate come da definire e saranno normate prima che le attività corrispondenti abbiano inizio.

1.2 Glossario

I termini specialistici saranno definiti nel *Glossario*, che il gruppo redigerà dopo l'aggiudicazione.

1.3 Riferimenti

Riferimenti normativi

- T. Vardanega, *Regolamento del Progetto Didattico*, a.a. 2026/2027, versione del 2026-10-11.

Riferimenti informativi

- ISO 8601, *Date and time – Representations for information interchange*.
- T. Preston-Werner, *Semantic Versioning 2.0.0*, `https://semver.org/lang/it/` (consultato il 2026-10-27).
- Verbali interni di Ampalis Team del 2026-10-19, del 2026-10-21 e del 2026-10-23.

2 Processi primari

2.1 Fornitura

Il processo regola i rapporti con il committente (i docenti) e con il proponente.

Comunicazioni ufficiali.

1. L'indirizzo ampalisteam@gmail.com è l'unico recapito ufficiale del gruppo verso docenti e proponenti: ogni comunicazione esterna parte da questo indirizzo.
2. Le consegne ai docenti avvengono per email, senza allegati, con il collegamento alla parte del repository che contiene i documenti consegnati e con la *Lettera di Presentazione* in evidenza.

Incontri con il proponente.

1. Il gruppo richiede l'incontro per email, inviando in anticipo ordine del giorno e domande.
2. Prima dell'incontro il gruppo stabilisce chi pone ciascuna domanda e chi redige il verbale.
3. Dopo l'incontro il redattore scrive il verbale esterno con il modello del gruppo; un altro componente lo verifica e il Responsabile lo approva.

4. Il verbale esce con il campo Conferma "In attesa di conferma" e viene inviato al proponente, che lo firma nell'ultima sezione.
5. Con la firma si rilascia una nuova versione del verbale, registrata nel registro delle modifiche: il campo Conferma riporta l'azienda e la data della conferma, l'ultima sezione riporta nome, ruolo, data e firma ricevuti. Il PDF firmato dall'azienda si conserva nel repository `documenti_bozza`.

2.2 Sviluppo

Da definire dopo l'aggiudicazione: analisi dei requisiti (compreso lo stato dell'arte richiesto dal proponente), progettazione, codifica, integrazione e rilascio.

3 Processi di supporto

3.1 Documentazione

Strumenti e modelli. I documenti si scrivono in LaTeX, con un preambolo comune e uno stile grafico ricavato dal logo (file `template/preambolo.tex`). Ogni nuovo documento parte da un modello della cartella `template`:

- `modello-documento.tex` per i documenti (per esempio *Valutazione dei Capitolati*, *Norme di Progetto*);
- `modello-verbale-interno.tex` e `modello-verbale-esterno.tex` per i verbali;
- `modello-diario-di-bordo.pptx` per le diapositive del diario di bordo.

Diapositive del diario di bordo. Le diapositive del diario di bordo si preparano in PowerPoint, con il modello del gruppo, e si caricano nel repository in formato PDF. LaTeX permetterebbe di produrre anche diapositive, ma per una presentazione breve, da preparare ogni settimana, PowerPoint è più rapido e permette a tutti i componenti di modificarla direttamente.

Struttura di un documento.

1. Copertina, con titolo, versione, data, stato, uso (interno o esterno), destinatari, redazione, verifica e approvazione, sempre con nome e cognome.
2. Registro delle modifiche, prima dell'indice.
3. Indice e, se il documento contiene tabelle, indice delle tabelle.
4. Corpo del documento: tabelle e figure sono numerate e hanno una didascalia.

Versionamento. Tutti i documenti, verbali compresi, hanno una versione secondo il versionamento semantico X.Y.Z e un registro delle modifiche. Fanno eccezione la *Lettera di Presentazione*, che ha solo la data perché accompagna una sola consegna e la versione inviata sostituisce le bozze, e il diario di bordo, che è un supporto all'esposizione in aula.

1. La prima stesura verificata ha versione 0.1.0; le modifiche verificate successive alzano Y (aggiunte o modifiche di contenuto) o Z (piccole correzioni).
2. Il primo rilascio, approvato dal Responsabile, ha versione 1.0.0. Dopo il rilascio, X sale con un nuovo rilascio che contiene modifiche grandi; Y e Z seguono la regola del punto 1.

3. Ogni modifica diventa una riga del registro solo dopo una verifica con esito positivo. La riga riporta versione, data, autore, verificatore e descrizione; le righe vanno dalla più recente alla più vecchia.
4. Autore e verificatore di una riga sono sempre persone diverse.

Verbali. Un verbale esce direttamente come versione 1.0.0, già verificato e approvato; ogni modifica successiva, verificata, aggiunge una riga al registro.

1. Ogni verbale riporta: informazioni sulla riunione, ordine del giorno, svolgimento, decisioni, azioni e prossima riunione.
2. Le decisioni hanno un codice (VI<n>-D<nn> nei verbali interni, VE<n>-D<nn> in quelli esterni) e riprendono quanto scritto nello svolgimento.
3. Ogni azione deriva da una decisione, ha assegnatario e scadenza ed è tracciata da una issue, il cui numero compare nel verbale.
4. Ogni verbale nuovo resta coerente con i precedenti: ciò che un verbale precedente ha già registrato si richiama, non si presenta come nuovo.

Lingua. Il repository e i documenti sono in italiano, lingua madre di tutti i componenti e lingua dei rapporti con il proponente: scrivere nella stessa lingua riduce le ambiguità all'interno del gruppo e con l'azienda. Restano in inglese i termini standard degli strumenti, che non si traducono: le etichette predefinite di GitHub (per esempio `bug`, `documentation`, `enhancement`) e i campi e le colonne della Project Board (per esempio Backlog, In progress, Done).

Date nel testo e nei messaggi di commit. Le date si scrivono nel formato AAAA-MM-GG, per esempio 2026-10-21 (formato esteso della norma ISO 8601).

- È un formato internazionale e non ambiguo: una data come 03/04/2027 si legge come 3 aprile in Italia e come 4 marzo nei paesi anglosassoni, mentre 2027-04-03 ha una sola lettura.
- Va dall'unità più grande alla più piccola, quindi l'ordine alfabetico coincide con quello cronologico, per esempio negli elenchi di commit e nelle tabelle.
- I trattini separano anno, mese e giorno e rendono la data leggibile a colpo d'occhio.

Nomi dei file. I nomi dei file sono in minuscolo, con le parole separate da trattini. Se il nome contiene una data, la data viene per prima, nel formato AAAAMMGG (formato base della norma ISO 8601, senza separatori), ed è separata dal resto del nome con un trattino basso: per esempio `20261021_verbale-interno.pdf`.

- L'ordine alfabetico dei file nelle cartelle e nel repository coincide con quello cronologico, come nel testo.
- Senza trattini la data è un unico blocco di cifre, distinto dal trattino che nei nomi separa le parole (`verbale-interno`) e dal trattino basso che separa la data dal resto del nome. Il nome resta più corto e la data si seleziona e si cerca come un'unica parola.
- Nel testo, al contrario, si usano i trattini, perché lì conta la leggibilità.

Regole di scrittura.

- I nomi dei documenti si scrivono in corsivo e con le iniziali maiuscole (per esempio *Dichiarazione degli Impegni*); in corsivo vanno anche i nomi di issue e milestone (per esempio *Candidatura*).

- Nei titoli di sezione va in maiuscolo solo la prima parola, salvo nomi propri e nomi di documenti; gli acronimi si sciolgono alla prima occorrenza.
- Il testo da completare o confermare si scrive in rosso; nessun testo rosso deve restare in un documento rilasciato.

3.2 Gestione della configurazione

Strumenti. Il gruppo usa git per il versionamento e GitHub per ospitare i repository, raccolti nella GitHub organization del gruppo (`https://github.com/Ampalis-Team/`). Tutto il materiale del progetto, bozze comprese, resta su GitHub: il gruppo non scambia documenti su altri canali, come Google Drive, WhatsApp o Discord.

Repository	Visibilità	Contenuto
`documentazione`	Pubblico	Documenti del progetto pronti per la verifica e rilasciati, consultabili da committente e proponente
`documenti_bozza`	Privato	Bozze dei documenti, condivise tra i componenti mentre sono in lavorazione, e materiale di lavoro, come i PDF dei verbali firmati dalle aziende
`progetto_ingegneria_del_software_2026-27`	Pubblico	Codice del Proof of Concept e del Minimum Viable Product; per ora è vuoto e ha un nome generico

Una bozza resta nel repository privato finché il suo autore la considera pronta; a quel punto entra nel repository `documentazione` con una pull request, dove viene verificata e approvata.

Per scelta del gruppo, nel repository `documentazione` i documenti si pubblicano già in PDF e non in sorgente LaTeX, così chi consulta il repository legge direttamente il documento finale. Fanno eccezione i modelli della cartella `template`, pubblicati anche in sorgente perché da essi partono i nuovi documenti.

Struttura del repository della documentazione. Il repository `documentazione` è organizzato in cartelle secondo il tipo e l'uso dei documenti, non secondo la consegna a cui appartengono: così un documento che evolve nel tempo resta sempre nella stessa cartella.

```
documentazione/
|-- documentazione-esterna/   # documenti per committente e proponente
|-- documentazione-interna/   # documenti a uso del gruppo
|-- template/                 # modelli dei documenti
|-- logo/                     # logo del gruppo
|-- vista-web/                # elenco e generatore della vista web
`-- .github/workflows/        # pubblicazione della vista web
```

- `documentazione-esterna` raccoglie i documenti rivolti all'esterno: i documenti delle candidature e delle revisioni (per esempio *Valutazione dei Capitolati*, *Dichiarazione degli Impegni*, *Lettera di Presentazione*), i verbali esterni e i diari di bordo.
- `documentazione-interna` raccoglie i documenti a uso del gruppo: i verbali interni, il *Way of Working* e, dopo l'aggiudicazione, le *Norme di Progetto*.
- `template` contiene i modelli dei documenti e dei verbali, in sorgente LaTeX e in PDF, da consultare senza compilarli.

- `logo` contiene il logo del gruppo, a colori e in bianco e nero.
- `vista-web` e `.github/workflows` contengono l'elenco dei documenti pubblicati, il generatore della vista web e la sua pubblicazione automatica.
- Nella radice, un file di presentazione descrive il repository e riporta l'indirizzo della vista web.

I nomi dei file seguono le regole della sezione 3.1.

Vista web. I documenti pubblicati si consultano anche dalla vista web del gruppo, `https://ampalis-team.github.io/documentazione/`, una pagina su GitHub Pages che li raccoglie per consegna, con la *Lettera di Presentazione* in evidenza, divisi in documentazione esterna e interna e con verbali e diari di bordo dal più recente.

- La pagina si rigenera a ogni modifica di `main` a partire dall'elenco dei documenti, il file `documenti.toml` nella cartella `vista-web`. Un documento compare quando il suo PDF è su `main`, cioè dopo che il Responsabile ha unito la pull request.
- La pull request che aggiunge un PDF, o ne pubblica una nuova versione, aggiorna anche la voce del documento nell'elenco: titolo, data e versione.
- Su ogni pull request un controllo automatico segnala i PDF non elencati, le voci incomplete e i nomi di file che non seguono le regole della sezione 3.1.

Rami. Il ramo `main` è protetto: si modifica solo con una pull request approvata da un componente diverso dall'autore; push forzati e cancellazione del ramo sono bloccati. Il lavoro si svolge in rami dedicati, che restano nel repository anche dopo l'unione, come storico.

Issue. Ogni attività è tracciata da una issue.

1. La issue ha uno o più assegnatari, le etichette pertinenti e la milestone a cui contribuisce.
2. Se nasce da un verbale, la descrizione riporta l'origine (verbale, azione e decisione) e la scadenza.
3. Ogni issue compare nella Project Board, dove se ne segue l'avanzamento (sezione 4.1).
4. Per quanto possibile, la issue si collega a ciò che la completa con una parola chiave di chiusura (per esempio `Closes #8`): alla pull request, se richiede più modifiche, oppure al singolo commit, se basta una modifica.

Pull request.

1. L'autore apre la pull request verso `main`, con un titolo che descrive il risultato e, nella descrizione, la parola chiave di chiusura della issue (per esempio `Closes #8`).
2. Come revisore indica il verificatore del documento e come assegnatario il Responsabile.
3. Il verificatore esamina le modifiche secondo la sezione 3.3 e, se l'esito è positivo, le approva.
4. Il Responsabile unisce la pull request: l'unione equivale all'approvazione del rilascio. La issue collegata si chiude in automatico.

Commit.

- Ogni componente fa i commit con il proprio account GitHub e con nome e indirizzo email associati a quell'account.

- Ogni commit contiene una sola aggiunta o modifica, piccola e circoscritta.
- Il messaggio è in italiano e descrive il risultato: un participio passato, poi l'oggetto, l'eventuale data nel formato AAAA-MM-GG e il riferimento alla issue. Esempio: "Redatto verbale interno 2026-10-21 Closes #8".

3.3 Verifica

1. Ogni prodotto è verificato da un componente diverso dal suo autore.
2. La verifica avviene nella revisione della pull request. Il verificatore controlla la correttezza del contenuto, il rispetto di questo documento e dei modelli, l'assenza di testo rosso e la coerenza con gli altri documenti, in particolare con i verbali precedenti. Controlla anche che la parola chiave di chiusura, nel commit e nella pull request, indichi la issue giusta.
3. Se trova difetti, il verificatore li segnala nella pull request e chiede le modifiche; l'autore le apporta e la verifica riprende dal punto 2.
4. Con esito positivo il verificatore approva la pull request; solo allora la modifica entra nel registro delle modifiche.

3.4 Gestione della qualità e validazione

Da definire dopo l'aggiudicazione, con il cruscotto di valutazione della qualità previsto dal regolamento e con il test book concordato con il proponente come criterio di accettazione del prodotto.

4 Processi organizzativi

4.1 Gestione organizzativa

Ruoli. Il gruppo adotta i ruoli del regolamento: Responsabile, Amministratore, Analista, Progettista, Programmatore e Verificatore. Nel periodo di candidatura il Responsabile è Ivan Valente, mentre redattori e verificatori dei documenti ruotano tra gli altri componenti. Dopo l'aggiudicazione i ruoli ruotano secondo le regole della *Dichiarazione degli Impegni*: un solo ruolo per persona alla volta, tutti i ruoli per ogni componente e nessuna verifica dei propri prodotti.

Comunicazione interna. Il gruppo usa Discord per le riunioni e un gruppo WhatsApp per le comunicazioni rapide.

Calendario condiviso. Riunioni, scadenze e indisponibilità sono raccolte in un calendario di Google Calendar creato appositamente per il gruppo.

1. A ogni evento sono invitati tutti i componenti, al loro indirizzo email istituzionale, per tenere separati gli impegni universitari da quelli privati.
2. Google Calendar avvisa tutti per email non appena un evento viene creato, modificato o cancellato, e con una notifica dell'app un giorno prima e un'ora prima dell'evento.
3. Ogni evento si ricorda anche nel gruppo WhatsApp, menzionando tutti i componenti.

Decisioni. Ogni decisione, anche la più piccola, si mette in pratica solo se tutti i componenti sono d'accordo, così da raccogliere sempre eventuali suggerimenti prima di agire.

1. Chi propone un'azione a nome del gruppo (per esempio l'invio di un'email) la scrive nel gruppo WhatsApp e menziona tutti i componenti con la funzione di menzione.
2. L'azione si svolge solo quando tutti hanno risposto in modo favorevole; finché manca anche una sola risposta, non si procede.
3. La stessa regola vale per le decisioni prese in riunione.

Riunioni interne.

1. Le riunioni interne si svolgono principalmente in chiamata online su Discord; almeno una riunione a settimana è fissa.
2. Le chiamate si registrano, così chi non riesce a partecipare può recuperarle e restare al passo.
3. Ogni riunione ha un ordine del giorno, che è il primo punto del verbale.
4. Un redattore, a rotazione, scrive il verbale interno con il modello del gruppo; un altro componente lo verifica e il Responsabile lo approva.
5. Le azioni decise in riunione diventano issue, con assegnatario e scadenza.

Metodo di lavoro. Il gruppo segue il metodo agile Scrum.

1. Le attività si raccolgono in una Project Board della GitHub organization, condivisa tra il repository della documentazione e quello del progetto: riguarda il gruppo nel suo insieme e raccoglie tutto il lavoro, dalla candidatura alla fine del progetto.
2. La Project Board usa un modello ispirato a Scrum, con vista Kanban e colonne Backlog, Ready, In progress, In review e Done.
3. Le attività si organizzano in milestone (per esempio *Candidatura*) e, dopo l'aggiudicazione, in sprint, con la durata definita nella *Dichiarazione degli Impegni*.

Definition of Done. Da definire. Finora il gruppo ha prodotto solo documenti, per i quali un'attività è conclusa quando il Responsabile approva il documento, dopo la verifica. Per il codice la *Definition of Done* sarà stabilita dopo l'aggiudicazione, in base alle indicazioni del proponente e alla realtà di interesse.

4.2 Formazione

Da definire dopo l'aggiudicazione: piano di formazione sulle tecnologie del capitolato, con un referente per ciascuna tecnologia.